Programming in Python
IDC-212
Lecture-4

August 15, 2025

Contents

1. Functions

2. Module

3. Packages

Functions

Why Use Functions in Python?

Motivation: When a block of instructions is needed multiple times, we can define it once as a function and call it whenever required. This promotes clean and reusable code.

Benefits of Functions:

- Encourages reusability and modular design.
- Avoids redundant code.
- Increases readability and maintenance.

Two Types of Functions in Python:

1. **Built-in Functions:**
   - Predefined by Python
   - Examples: `id()`, `type()`, `eval()`, `input()`
2. **User-Defined Functions:**
   - Created by programmers for specific tasks.

User-Defined Functions in Python

Syntax of a Function:

```python
def function_name(parameters):
    """Docstring explaining the function"""
    # Body of the function
    return value  # optional
```

Example: Greeting Function for IISER Lab

```python
def greet_iiser():
    print("Hello Einstein, Welcome to IISER Physics Lab!")
```

Note:

- `def` is mandatory.
- `return` is optional.
- Docstring is optional but good practice.

Function Parameters: Introduction

Parameters are inputs to a function. When a function has parameters, we must supply arguments when calling it, else a runtime error will occur.

Example: Print a greeting message with name

```python
def greet(scientist):
    print("Hello", scientist, "Good Morning")

greet("iiser")
greet("Raman")
```

Expected Output:

- Hello iiser Good Morning
- Hello Raman Good Morning

Functions with Parameters and Return Values

Example 1: Square of a number

```python
def square_of(number):
    print("The square of", number, "is", number * number)

square_of(4)
```

Output:

- The square of 4 is 16

Example 2: Sum of two numbers using return

```python
def add_numbers(x, y):
    return x + y

result = add_numbers(10, 20)
print("The Sum is", result)
```

Output: The Sum is 30

No Return Statement Example

Note: If no `return` is used, Python returns `None` by default.

```python
def say_hello():
    print("Hello from IISER!")

say_hello()
print(say_hello())
```

Output:

- Hello from IISER!
- Hello from IISER!
- None

Returning Multiple Values from a Function

Why multiple return values?

In Python, unlike C/C++ or Java, functions can return more than one value. This is useful when we want to compute and retrieve multiple results at once.

Example 1: Return sum and difference of two values

```python
def compute_results(a, b):
    addition = a + b
    difference = a - b
    return addition, difference
# Call the function and unpack the results
sum_res, diff_res = compute_results(120, 45)
print("Sum is:", sum_res)
print("Difference is:", diff_res)
```

Output:

- Sum is: 165
- Difference is: 75

Returning Multiple Values (Contd...)

Example 2: Return all basic operations on two values

```python
def operations(a, b):
    total = a + b
    diff = a - b
    prod = a * b
    div = a / b
    return total, diff, prod, div
result = operations(150, 30)
print("Results:")
for value in result:
    print(value)
```

Output:

- 180
- 120
- 4500
- 5.0

Types of Arguments in Python

Python supports 4 types of actual arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Length Arguments

1) Positional Arguments

- Arguments are passed in the correct order.
- The number and position must match.

```python
def subtract(a, b):
    print(a - b)

subtract(100, 200)   # Output: -100
subtract(200, 100)   # Output: 100
```

Keyword Arguments in Python

2) Keyword Arguments

- Values are assigned using the parameter names.
- Order doesn't matter when using keywords.

```python
def greet(name, msg):
    print("Hello", name, msg)

greet(name="iiser", msg="tvm")
greet(msg="tvm", name="iiser")
```

Output:

- Hello iiser tvm
- Hello iiser tvm

Mixing Positional and Keyword Arguments

```python
def greet(name, msg):
    print("Hello", name, msg)

greet("iiser", "Good Morning")          # Valid
greet("iiser", msg="Good Morning")      # Valid
greet(name="iiser", "Good Morning")     #     Invalid
```

Rule: You must write positional arguments before keyword arguments. Otherwise, it raises: SyntaxError: positional argument follows keyword argument

Default Arguments

You can provide default values for parameters. If the caller omits the argument, the default is used.

```
def greet(name="Guest"):
    print("Hello", name, "Good Morning")

greet("Ronaldo")     # Hello Ronaldo Good Morning
greet()               # Hello Guest Good Morning
```

Only the missing arguments use the default values.

Important Rule for Default Arguments

Default arguments should be placed last. Non-default arguments after default ones will cause error.

```
#      Valid
def greet(name="iiser", msg="Good Morning"):
    print("Hello", name, msg)

#      Valid
def greet(name, msg="Good Morning"):
    print("Hello", name, msg)

#      Invalid
def greet(msg="Good Morning", name):
    print("Hello", name, msg)
```

Error: SyntaxError: non-default argument follows default argument

4) Variable Length Arguments in Python (Part 1)

- These are called **variable length arguments**.
- We use an asterisk (*) to define a variable-length parameter.

```python
def iiser_sum(*n):
    total = 0
    for n1 in n:
        total = total + n1
    print("The Sum =", total)
```

```python
iiser_sum()
iiser_sum(10)
iiser_sum(10, 20)
iiser_sum(10, 20, 30, 40)
```

```
The Sum= 0
The Sum= 10
The Sum= 30
The Sum= 100
```

Variable Length Arguments in Python (Part 2)

Mixing with Positional Arguments:

- We can mix variable-length arguments with positional arguments.
- In such cases, positional arguments must come first.

```python
def f1(n1, *s):
    print(n1,end=' ')
    for s1 in s:
        print(s1,end=' ')
    print()
f1(10)
f1(10, 20, 30, 40)
f1(10, "A", 30, "B")
```

Output:

```
10
10 20 30 40
10 A 30 B
```

Variable Length Arguments with *args

- We can mix *args with other positional arguments.
- If we want to pass values after *args, they must be passed as keyword arguments.

```python
def f1(*s, n1):
    for s1 in s:
        print(s1)
    print(n1)

f1("A","B",n1=10)       # Valid
f1("A","B",10)          # Invalid: TypeError
```

Output

```
A
B
10
TypeError: f1() missing 1 required keyword-only argument: 'n1'
```

Keyword Variable Length Arguments with **kwargs

- We can pass any number of keyword arguments using **kwargs.
- These keyword arguments are stored as a dictionary.
- Syntax:

```
def f1(**n):
```

```python
def display(**kwargs):
    for k, v in kwargs.items():
        print(k, "=", v,end= ' ')
    print()
display(n1=10, n2=20, n3=30)
display(rno=100, name="Ravi", marks=70, subject="Chemistry")
```

Output

```
n1 = 10 n2 = 20 n3 = 30
rno = 100 name = Ravi marks = 70 subject = Chemistry
```

Exercise: Valid Function Calls with Defaults

Function Definition:

```python
def display_info(arg1, arg2, arg3=4, arg4=8):
    print(arg1, arg2, arg3, arg4)
```

Valid Function Calls:

- `display_info(3, 2)` $\Rightarrow$ 3 2 4 8
- `display_info(10, 20, 30, 40)` $\Rightarrow$ 10 20 30 40
- `display_info(25, 50, arg4=100)` $\Rightarrow$ 25 50 4 100
- `display_info(arg4=2, arg1=3, arg2=4)` $\Rightarrow$ 3 4 4 2

Exercise: Invalid Function Calls (Missing/Ordering Errors)

Invalid Function Calls (TypeError):

- `display_info()`
  `TypeError: missing 2 required positional arguments: 'arg1' and 'arg2'`
- `display_info(arg3=10, arg4=20, 30, 40)`
  `SyntaxError: positional argument follows keyword argument`
  [After keyword arguments, we cannot pass positional arguments.]

Exercise: Invalid Calls (Duplicate or Extra Arguments)

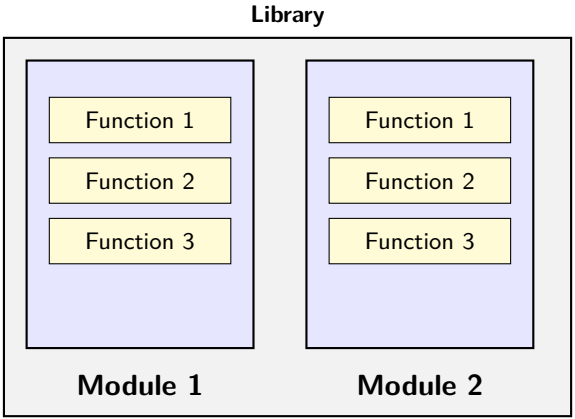
More Invalid Calls:

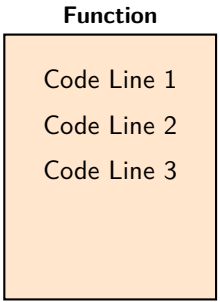
- `display_info(4, 5, arg2=6)`
  `TypeError: got multiple values for argument 'arg2'`
- `display_info(4, 5, arg3=5, arg5=6)`
  `TypeError: got an unexpected keyword argument 'arg5'`

Function vs Module vs Library

- **Function:** A block of code with a specific name that performs a task.
- **Module:** A file containing related functions, variables, and classes.
- **Library:** A collection of modules packaged together.

Function vs Module vs Library





Types of Variables in Python

Python supports 2 types of variables:

1. Global Variables
2. Local Variables

1) Global Variables

- Variables declared outside all functions.
- Accessible throughout the module.

```python
a = 10  # global variable
def f1():
    print(a)

def f2():
    print(a)

f1()
f2()
```

Output:

```
10
10
```

2) Local Variables

- Declared inside a function.
- Only accessible within the function scope.

```python
def f1():
    a = 10
    print(a)  # valid

def f2():
    print(a)  # invalid

f1()
f2()
```

Output:

```
10
NameError: name 'a' is not defined
```

global Keyword: Purpose

We can use `global` keyword for:

1. Declaring a global variable inside a function.
2. Making a global variable modifiable inside a function.

Declaring Global Inside Function

```python
def f1():
    global a
    a = 777
    print(a)

def f2():
    print(a)

f1()
f2()
```

Output:

```
777
777
```

Modifying Global Variable

```python
a = 10

def f1():
    global a
    a = 777
    print(a)

def f2():
    print(a)

f1()
f2()
```

Output:

777
777

Name Conflict: Local vs Global

Note: If a global and a local variable have the same name, you can still access the global variable using `globals()`.

```python
a = 10  # Global Variable

def f1():
    a = 777  # Local Variable
    print(a)
    print(globals()['a'])

f1()
```

Output:

```
777
10
```

Anonymous Functions and Lambda Expressions

Anonymous Functions:

- We can declare a function without any name. Such nameless functions are called **anonymous functions** or **lambda functions**.
- The main purpose is instant or one-time use.

Normal Function:

```
def squareIt(n):
    return n * n
```

Lambda Function Syntax:

- lambda argument_list : expression
- Example: `lambda n:  n * n`

Note: Lambda functions help write concise and readable code.

Q) Lambda Function to Find Square of a Number

```python
s = lambda n: n * n
print("The Square of 4 is:", s(4))
print("The Square of 5 is:", s(5))
```

Output:

- The Square of 4 is: 16
- The Square of 5 is: 25

Q) Lambda Function to Find Sum of Two Numbers

```python
s = lambda a, b: a + b
print("The Sum of 10, 20 is:", s(10, 20))
print("The Sum of 100, 200 is:", s(100, 200))
```

Output:

- The Sum of 10, 20 is: 30
- The Sum of 100, 200 is: 300

Q) Lambda Function to Find Maximum of Two Numbers

```python
s = lambda a, b: a if a > b else b
print("The Biggest of 10, 20 is:", s(10, 20))
print("The Biggest of 100, 200 is:", s(100, 200))
```

Output:

- The Biggest of 10, 20 is: 20
- The Biggest of 100, 200 is: 200

filter() Function in Python

- We can use `filter()` function to filter values from the given sequence based on a condition.
- Syntax: `filter(function, sequence)`
- The function checks each item and returns only those that meet the condition.

Example: Filter Even Numbers (Without Lambda)

```python
def isEven(x):
    if x % 2 == 0:
        return True
    else:
        return False

l = [0, 5, 10, 15, 20, 25, 30]
l1 = list(filter(isEven, l))
print(l1)  # [0, 10, 20, 30]
```

filter() with Lambda

Example: Using Lambda Expression

```
l = [0, 5, 10, 15, 20, 25, 30]
l1 = list(filter(lambda x: x % 2 == 0, l))
print(l1)  # [0, 10, 20, 30]

l2 = list(filter(lambda x: x % 2 != 0, l))
print(l2)  # [5, 15, 25]
```

map() Function in Python

- The `map()` function applies a function to each item in a sequence.
- Syntax: `map(function, sequence)`

Without Lambda:

```
l = [1, 2, 3, 4, 5]
def doubleIt(x):
    return 2 * x
l1 = list(map(doubleIt, l))
print(l1)  # [2, 4, 6, 8, 10]
```

map() with Lambda

Using Lambda Function

```
l = [1, 2, 3, 4, 5]
l1 = list(map(lambda x: 2 * x, l))
print(l1)  # [2, 4, 6, 8, 10]

# Square of each number
l1 = list(map(lambda x: x * x, l))
print(l1)  # [1, 4, 9, 16, 25]
```

map() with Multiple Lists

Example: Multiply Elements from Two Lists

```
l1 = [1, 2, 3, 4]
l2 = [2, 3, 4, 5]
l3 = list(map(lambda x, y: x * y, l1, l2))
print(l3)  # [2, 6, 12, 20]
```

Everything is an Object in Python

Key Concepts:

- In Python, everything is treated as an object.
- Even functions are internally treated as objects.

Example Code:

```python
def f1():
    print("Hello")

print(f1)
print(id(f1))
```

Output:

- `<function f1 at 0x00419618>`
- `4298264`

Function Aliasing in Python

Definition: Assigning an existing function to a new name is called function aliasing.

Example Code:

```python
def wish(name):
    print("Good Morning:", name)

greeting = wish
print(id(wish))
print(id(greeting))

greeting('iiser')
wish('iiser')
```

Output:

- 4429336
- 4429336
- Good Morning: iiser
- Good Morning: iiser